SIMATS ENGINEERING

REVERSE ENGINEERING TASK

SOFTWARE TESTING

Student Name	Bhavani Sankar
Register Number	192411238
Course Code	CSA3712
Assessment	CO2 – AT2 Reverse Engineering Task
Course Outcome	Design effective test cases using black-box and white-box testing techniques and control methods flow.

Submitted by: Bhavani Sankar

Register Number: 192411238

ANSWERS TO SCENARIO-BASED QUESTIONS

Note: The supplied assessment describes scenarios but does not include the referenced source-code snippets, execution traces, API JSON examples, logs, or profile values. Where these are absent, the answers provide conceptual models and identify what must be confirmed from the actual system.

1. ATM Withdrawal Module: Reverse Engineering Black-Box Tests

System Decomposition

The ATM withdrawal function can be divided into card and PIN validation, account selection, amount entry, balance checking, withdrawal-limit checking, cash dispensing, account update, receipt generation, and error handling. Each component should be tested separately and then as part of a complete transaction.

Inferred Equivalence Classes

For withdrawal amount, valid inputs are positive amounts within the permitted per-transaction limit and available balance. Invalid classes include zero, negative amounts, non-numeric values, values below the minimum denomination, amounts above the transaction limit, and amounts greater than the available balance. Other classes include valid/invalid PIN, active/blocked card, and available/unavailable ATM cash.

Boundary Conditions

Let the permitted withdrawal range be L to U, inclusive. Test L–1, L, L+1, a typical amount, U–1, U, and U+1, using the actual configured limits. Also test the account balance just below, equal to, and just above the requested amount, and daily-limit values at and around their boundaries.

Reconstructed Test Cases

Test a valid withdrawal; invalid PIN; blocked card; zero or negative amount; amount below minimum denomination; amount at the minimum and maximum; amount above the maximum; insufficient balance; daily limit exceeded; ATM cash unavailable; network timeout; and a duplicate request after timeout. Verify that failed transactions do not debit the account and that successful withdrawals update the balance exactly once.

Adequacy and Improvement

Observed input-output tests suggest black-box functional testing, but the limited evidence cannot prove which exact techniques were originally used. Improve the suite by applying equivalence partitioning, boundary value analysis, decision tables for account/card/cash conditions, and negative tests. Add regression tests and verify transaction logs, rollback, and receipt correctness.

2. Execution Traces and Code Snippets: Reconstructing White-Box Tests

Control-Flow Graph Model

Because the assessment does not provide actual code snippets or execution traces, the following is a conceptual CFG: Start → initialize inputs → validate input [valid?]. If invalid, return error → End. If valid, enter processing loop [more items?]. For each item, evaluate condition A [true/false], perform the corresponding operation, then evaluate condition B [true/false]. After the loop ends, finalize result → End.

Decision Points and Loops

The decision points are input validity, loop continuation, condition A, and condition B. The loop has an entry, repeated body, and exit edge. When actual code is available, every if/else, switch case, exception branch, and loop condition should be mapped to a node or edge in the CFG.

Independent Paths

A conceptual basis set includes: (1) invalid input and immediate exit; (2) valid input with zero loop iterations where permitted; (3) valid input with one iteration and condition A true; (4) one iteration with condition A false; (5) paths exercising both outcomes of condition B; and (6) multiple iterations followed by normal termination. Exact independent paths and cyclomatic complexity require the missing source code.

Coverage Evaluation

Statement coverage proves that statements ran at least once but may miss false branches. Branch coverage requires both outcomes of each decision. Basis-path testing exercises a set of linearly independent paths. Full path coverage may not be feasible for loops or complex logic, so critical paths and loop boundaries should be prioritized.

Reconstructed Strategy

Create tests from the real CFG, execute every decision outcome, test loop counts of zero, one, two, and maximum supported iterations, and cover error/exception paths. Calculate V(G)=E−N+2P, normally with P=1 for a single connected graph, to estimate the minimum number of independent paths. Use a coverage tool and link every test to a requirement.

3. Login and Transaction System: Hybrid Test Reconstruction

Component Decomposition

Separate the system into credential validation, authentication/OTP, session management, account permissions, transaction input validation, balance and limit checks, transaction processing, database update, notification, and audit logging.

Likely Techniques from Defect Patterns

Defects concentrated at numeric limits may suggest incomplete boundary value analysis. Failures involving invalid formats may indicate weak equivalence partitioning. Errors that appear only after a sequence of decisions may indicate insufficient branch or path testing. These are hypotheses only; defect logs alone cannot confirm the original technique.

Black-Box Test Design

For login, test valid and invalid usernames/passwords, blank fields, malformed input, locked accounts, expired OTPs, reused OTPs, and rate-limit boundaries. For transactions, test minimum/maximum amounts, zero/negative amounts, insufficient funds, exceeded daily limits, invalid recipients, and network failures.

White-Box Test Design

Construct CFGs for login authorization and transaction processing. Cover true and false outcomes for password validity, OTP validity, account status, balance sufficiency, and transaction-limit checks. Test independent paths through success, denial, timeout, rollback, and recovery.

Integrated Strategy

Use ECP and BVA for inputs, decision tables for combinations of account status and transaction rules, and basis-path testing for internal logic. Add integration, concurrency, security, and regression testing. Verify that failed transactions leave balances unchanged and that successful transactions are recorded once. Prioritize high-severity defects and keep traceability between requirements and tests.

4. Inventory Management: Reverse Engineering Stock Logs

Functional Decomposition

The stock update process can be divided into receiving transaction input, validating product ID and quantity, checking current stock, applying stock movement, recalculating inventory, checking reorder thresholds, generating low-stock alerts, and recording the transaction.

Inferred Equivalence Partitions

For quantity, likely classes include positive valid quantity, zero, negative quantity, missing quantity, and non-numeric quantity. For stock level, classes include below reorder point, exactly at reorder point, above reorder point, zero stock, and insufficient stock for a requested issue. Logs can suggest these classes, but the exact business rules must be confirmed.

Boundary Tests

If R is the reorder threshold, test stock levels R−1, R, and R+1. For stock issues, test requested quantity one less than current stock, equal to current stock, and one greater than current stock. Test zero and negative quantities, the minimum supported unit, and maximum allowed transaction size.

Reconstructed Test Suite

Test valid stock receipt, valid stock issue, issue that leaves stock at the reorder threshold, issue that crosses below the threshold, issue with insufficient stock, duplicate transaction, unknown product ID, and database failure. Verify both the stored stock balance and whether the low-stock alert is generated exactly when required.

Justification

Logs reveal observed outcomes but may not expose untested rules. Combine ECP, BVA, decision-table tests for receipt/issue and alert conditions, and negative tests. Add regression cases for every previously observed defect and reconcile transaction logs with the final stock balance.

5. Compiled Library Module: Resource Profiles and Path Testing

Interpreting CPU and Memory Spikes

High CPU use may correspond to repeated searches, sorting, nested loops, or repeated retries. Memory spikes may suggest large collections, unbounded buffering, object retention, or loops that accumulate results. A profile alone cannot identify exact source-level paths; it guides investigation.

Conceptual CFG

Start → load request → validate query [valid?]. If invalid, return error. If valid, fetch records → loop over records [more records?] → validate each record [valid?] → update result list. After the loop, sort or format results → return response. Add explicit branches for empty results, large datasets, exceptions, and resource-limit failures.

Targeted Test Strategy

Test empty, small, typical, and maximum supported libraries; searches with no match, one match, and many matches; invalid queries; repeated requests; and simultaneous requests. Test loop boundaries and large result sets while recording execution time and memory consumption.

Path and Stability Evaluation

Use CFG-based branch and basis-path tests for validation, empty/non-empty results, per-record valid/invalid paths, and error handling. Add stress, endurance, and load tests with thresholds for response time and memory. Use profiling to confirm whether a suspected path causes resource spikes, and check that resources are released after completion.

Limitations

The assessment mentions a compiled executable and profile but supplies no actual profile values or code. Therefore, the CFG is conceptual; exact nodes, independent paths, and complexity must be derived from the real executable analysis or source-level instrumentation.

6. Mobile Payment API: State Transition Testing

Inferred State Machine

The named transaction states are Pending, Success, Reversal, and Timeout. A conceptual model is: request accepted → Pending; Pending → Success when confirmation arrives; Pending → Reversal when a failed/uncertain debit must be undone; Pending → Timeout when no response arrives within the allowed period. A late success after timeout should follow a defined reconciliation rule rather than silently creating a duplicate payment.

Transition Table

Pending→Success: valid success response and transaction reference. Pending→Reversal: failure or duplicate debit requiring compensation. Pending→Timeout: response deadline expires. Timeout→Success: accept only if the specification permits late confirmation and reconciliation is performed. Success→Reversal: allow only for a legitimate reversal event. Reversal→Success should normally be rejected unless the business specification defines a controlled retry/new transaction.

Test Cases

Test each legal transition, each forbidden transition, repeated callbacks, duplicate request IDs, delayed responses, malformed JSON, missing transaction IDs, and inconsistent amounts/currency. Verify idempotency so retries do not charge twice and every reversal references the original transaction.

Coverage and Data Constraints

Check that required JSON fields exist, transaction IDs are unique, amounts are valid, and status values belong to the allowed set. Cover every state and transition, including timeout recovery and duplicate events. The exact legal transition set must be confirmed against the API specification.

State Diagram in Text

Start → Pending → Success. From Pending, the transaction may instead move to Reversal or Timeout according to the response and timeout rules. A permitted late confirmation from Timeout enters reconciliation and must end in a consistent final state. Invalid transitions are rejected and logged.

7. Encrypted Communication: Fuzzing and Robust BVA

Component Decomposition

Separate input parsing and validation, length and format checks, key/nonce handling, encryption, output encoding, and error handling. Input validation should occur before data is passed to cryptographic routines.

Fuzzing Targets

Generate malformed, missing, oversized, truncated, and unexpected-type values for message length, key identifier, nonce/initialization vector, algorithm selection, and encoded payload. Include invalid encodings, embedded nulls, boundary lengths, and repeated requests. Use an authorized isolated test environment.

Boundary Value Analysis

For each documented length range [L,U], test L–1, L, L+1, a typical value, U–1, U, and U+1. Also test empty input and maximum supported payload. For fixed-size cryptographic fields, test one byte short, exact length, and one byte long. Use only the actual algorithm's specified sizes.

Security Validation

Confirm oversized or malformed data is rejected safely, without crashes, memory corruption, secret leakage, or partial output. Verify that errors do not reveal key material and that invalid inputs cannot bypass authentication or authorization. Use memory-safety tools and sanitizers where available.

Justification

Fuzzing explores unexpected input combinations, while BVA systematically targets length boundaries. Together they help identify parser errors, buffer handling defects, and injection risks. Testing can reduce risk but does not by itself prove the absence of vulnerabilities; code review and security analysis are also required.

8. Insurance Premium Calculator: Statement vs Branch/Path Coverage

Original Approach and Gap

100% statement coverage means every executable statement ran, but a compound if condition can still have untested true/false outcomes for individual conditions. Therefore, statement coverage alone is inadequate for a premium calculator with multi-condition rules.

Conceptual CFG

Start → validate age and policy type → decision: inputs valid? No → error → End. Yes → decision: age condition satisfied? → decision: risk category eligible? → calculate base premium and apply adjustments → validate final amount → return premium. Each decision must have true and false edges; invalid inputs and calculation exceptions need explicit paths.

Independent Paths

A basis set should include invalid input; valid input with age condition true and risk condition true; age true/risk false; age false/risk true; and age false/risk false, where those combinations are allowed. Add paths for each policy type and for final amount validation. Exact paths depend on the actual source code.

Improved Coverage

Achieve branch coverage by exercising every decision outcome. Use condition coverage or MC/DC for compound Boolean expressions where risk warrants it. Basis-path testing uses cyclomatic complexity V(G)=E–N+2P to guide independent path selection. Full path coverage may not be feasible if loops or numerous combinations exist, so state any constraints clearly.

Impact on Calculation Quality

Test age boundaries, policy limits, discounts, surcharges, rounding, missing data, and invalid combinations using ECP and BVA. Compare outputs with independently calculated expected premiums. This catches logic combinations that statement coverage can miss and reduces incorrect quotes and billing.

9. Cloud File Synchronization: ECP, BVA, and Stress Testing

Defect Pattern Analysis

Lost-file defects associated with file size, file type, or network speed suggest that some input classes or boundary conditions were not adequately tested. Activity logs can help correlate loss with large transfers, unsupported formats, interrupted connections, or concurrent updates.

Likely Equivalence Classes

File-size classes may include empty, small, typical, near-limit, over-limit, and very large files. File-type classes may include supported formats, unsupported formats, files with unusual extensions, and files with misleading content types. Network classes may include stable, slow, intermittent, and disconnected connections. These are proposed classes to validate against product requirements.

Boundary and Stress Tests

Test sizes just below, at, and just above the documented upload limit. Test zero-byte files, files near chunk-size boundaries, and files that span multiple chunks. Simulate slow networks, connection loss during transfer, retries, simultaneous edits, and disk-space exhaustion.

Reconstructed Strategy

Combine ECP and BVA with stress and endurance tests. Verify checksums, file counts, metadata, version history, retry idempotency, and conflict resolution. Confirm that a partial transfer is not reported as complete and that a failed sync can resume without losing or duplicating data.

Justification

Input partitions identify representative cases, boundaries expose off-by-one and chunking errors, and stress tests exercise resource and concurrency limits. Together with recovery tests, logging, and regression automation, this strategy reduces the likelihood of future data loss.

10. E-Commerce Search and Filter: Decision Table Testing

Reverse-Engineering Search Logic

Inspect query parameters and result sets to infer whether filters are combined using AND, OR, and NOT. For example, category AND in-stock may require both conditions; category OR brand may return records matching either; NOT discontinued may exclude discontinued products. The actual SQL and observed results should confirm the rules.

Test Conditions

Define conditions for keyword match, category match, brand match, price range, stock status, and exclusion flag. For each condition, use true, false, and where appropriate not supplied. Define expected results for combinations based on the intended business rule.

Decision Table

Create rows for important combinations: keyword AND category; category AND in-stock; brand OR category; price range AND stock; excluded category with otherwise matching product; no filters; conflicting filters; and empty result set. For each row, record expected result IDs or expected count. Test both AND/OR precedence and parentheses.

Negative and Boundary Cases

Test minimum and maximum price, reversed price ranges, empty keyword, special characters, unknown category, missing parameters, duplicate filters, pagination boundaries, and sorting. Ensure user input is parameterized and cannot alter SQL structure.

Completeness Evaluation

A decision table makes combinations and expected outputs explicit. If the number of combinations is large, use pairwise testing for lower-risk conditions while retaining tests for high-risk interactions and operator precedence. Compare actual result sets against a trusted expected dataset and add regression tests for each defect.